SIMATS ENGINEERING

ASSESSMENT TOOL 2

Technical Presentation

Course Name	Internet Programming
Course Code	CSA4307
Course Outcome	CO 3 (L3)
Weightage	20%
Total Marks	20

Student Name	K Ganesh
Register Number	192411170

1. Servlet Architecture & Workflow

Question: Present the architecture of Java Servlets and explain the complete request-response cycle with a neat diagram.

Focus Areas: Client-Server interaction, Role of Web Container, Request and Response objects

Answer

a) Client-Server interaction

A Java servlet is a server side program that runs inside a web server and generates dynamic web pages. The architecture follows the client-server model. The client is the web browser, which sends a request using the HTTP protocol. The server side has the web server with the servlet container, and the servlet class which does the actual processing. If the application needs data, the servlet also talks to a database. The servlet never talks to the browser directly. Everything goes through the web container.

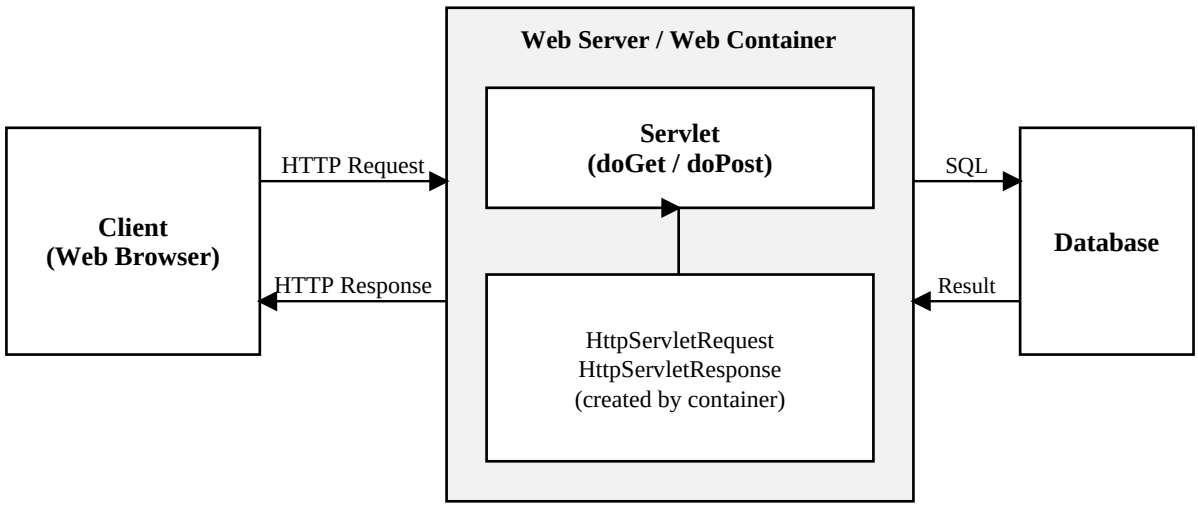

Figure 1: Architecture of a Java servlet application

b) Role of the web container

The web container (for example Apache Tomcat) is the part of the server that manages the servlets. It receives the HTTP request from the client and finds out which servlet should handle it, using the URL pattern given in web.xml or in the @WebServlet annotation. It loads the servlet class, creates the object and controls the life cycle by calling init(), service() and destroy(). For every request it creates a new thread, so many users can use the same servlet at the same time. It also provides services like session management, security and the conversion of the response into an HTTP response.

c) Request and response objects

For every client request the container creates two objects and passes them to the servlet. The HttpServletRequest object carries the data sent by the client, such as form parameters (getParameter()), headers, cookies and the session. The HttpServletResponse object is used by the servlet to send the output back. With it the servlet sets the content type (setContentType()), writes the HTML using getWriter(), adds cookies or redirects the user.

d) Complete request-response cycle

1. The user types the URL or submits a form in the browser, and the browser sends an HTTP request to the web server.
2. The web container receives the request and finds the servlet mapped to that URL.
3. The container creates the HttpServletRequest and HttpServletResponse objects. If the servlet is not yet loaded, it loads it and calls init().
4. The container calls the service() method on a new thread, which calls doGet() or doPost() according to the request type.
5. The servlet reads the request data, does the processing (it may connect to the database) and writes the result into the response object.
6. The container converts the response into an HTTP response and sends it to the browser, which displays the page. The request and response objects are then destroyed.

2. Servlet Life Cycle Analysis

Question: Explain the Servlet Life Cycle with a sequence diagram and analyze the role of init(), service(), and destroy() methods.

Focus Areas: Life cycle phases, Execution flow, Performance implications

Answer

a) Life cycle phases

The life cycle of a servlet is fully managed by the web container. It has four phases: loading and instantiation, initialization, request handling and destruction. In the first phase the container loads the servlet class and creates one object of it. This happens either when the first request comes or when the server starts, if load-on-startup is given. After this, the init(), service() and destroy() methods are called in the order shown in the sequence diagram.

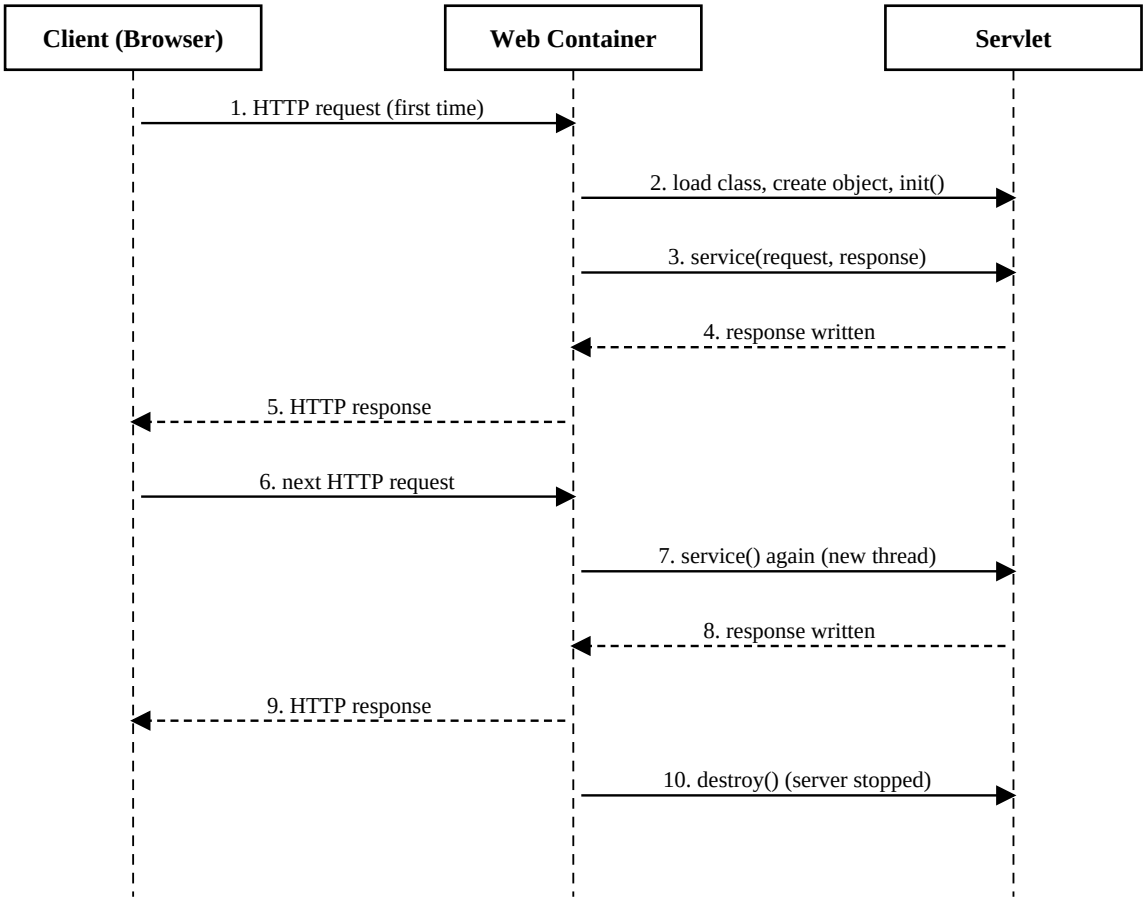

Figure 2: Sequence diagram of the servlet life cycle

b) Role of init(), service() and destroy()

Method	Role	Called
init()	Initializes the servlet. Used for one time work such as reading init parameters, loading a driver or creating a database connection	Once, after the object is created
service()	Receives the request and response objects, checks the HTTP method and calls doGet(), doPost() etc.	Every time a request comes, on a separate thread
destroy()	Releases the resources such as database connections, files and threads before the servlet is removed	Once, when the servlet is unloaded or the server stops

c) Execution flow

The container creates the servlet object only once and the same object is used for all the requests. First init() runs and it must finish successfully before the servlet can take any request. After that, each request from a client creates a new thread, and that thread calls service(). When the server is shut down or the application is undeployed, the container waits for the running requests to finish and then calls destroy(). After destroy() the object is garbage collected.

d) Performance implications

Because init() runs only once, the costly work like opening a database connection or loading a large file can be done there and used by all the requests. This makes the servlet faster than the old CGI programs, where a new process was created for every request. The container uses threads, which are lighter than processes, so the memory and CPU use is low. But since a single object is shared by many threads, the instance variables of a servlet are not thread safe. Shared data should be handled with local variables or synchronization, and too much synchronization will slow the application. The code inside service() should also be short and quick, because a slow request keeps the thread busy. Finally, destroy() must close the resources properly, otherwise there will be memory and connection leaks.

3. GET vs POST Method Evaluation

Question: Compare GET and POST methods used in form submission. Present scenarios where each method is appropriate.

Focus Areas: Data transmission, Security aspects, URL limitations

Answer

a) Comparison

Point	GET	POST
Data transmission	Data is added to the end of the URL as a query string (name=value pairs)	Data is sent inside the body of the HTTP request
Visibility	Visible in the address bar, browser history and server logs	Not visible in the URL
Security	Less secure, not suitable for passwords or personal data	More secure than GET, but HTTPS is still needed for encryption
URL limitation	Limited length of URL (about 2000 characters in most browsers), only text	No such limit, can send large data and files
Bookmark	Page can be bookmarked and shared	Cannot be bookmarked
Caching	Can be cached by the browser	Not cached normally
Repeat request	Safe to refresh, it only reads the data (idempotent)	Browser warns before resending, it may change the data again
Servlet method	doGet()	doPost()
Used for	Getting or searching the data	Sending or changing the data

b) Data transmission, security and URL limitation

In the GET method the form data goes along with the URL, for example *search?item=laptop&price=40000*. Since the data is part of the URL it can be seen by anyone who is near the screen and it is also saved in the history and in the server logs, so it is not secure. The length of the URL is also limited, so a large form or a file cannot be sent. In the POST method the data is placed in the body of the request. It does not appear in the URL, there is no size limit, and even binary data like images can be uploaded. Neither of them encrypts the data by itself, so HTTPS is required in both cases for real security.

c) Scenarios

Method	Appropriate scenarios
GET	Searching for a product on a shopping site, opening a product details page, filtering and sorting a list, sharing the link of a page, viewing the result of a student by giving the register number in the link
POST	Login with username and password, new user registration, online payment, submitting an application form, uploading a photo or a file, changing the profile details

To sum up, GET should be used when the request only reads the data and nothing secret is sent, and POST should be used when the data is sensitive, large, or when the request changes something on the server.

4. Session Tracking Techniques

Question: Present different session tracking mechanisms (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas: Session persistence, Security concerns, Use-case comparison

Answer

HTTP is a stateless protocol, so the server does not remember a user between two requests. Session tracking is the method used to identify the same user across many requests. The main techniques in servlets are given below.

a) HttpSession

The container creates a session object on the server and gives it a unique session ID (JSESSIONID), which is sent to the browser. The data of the user is kept on the server and only the ID travels between the client and the server.

b) Cookies

A cookie is a small text file stored in the browser by the server. The browser sends it back automatically with every request to the same site, so the server can recognise the user.

c) URL rewriting

The session ID or any data is added to the end of every URL, for example *profile;jsessionid=A1B2C3*. In servlets it is done using response.encodeURL(). It is used when the browser has disabled cookies.

d) Hidden form fields (additional)

The data is kept in hidden fields of the HTML form and it is sent again when the form is submitted. It works only in form based pages.

```
// HttpSession
HttpSession session = request.getSession();
session.setAttribute("user", "ganesh");

// Cookie
Cookie c = new Cookie("theme", "dark");
c.setMaxAge(60 * 60 * 24);
response.addCookie(c);

// URL rewriting
String url = response.encodeURL("profile");

// Hidden field (in HTML)
// <input type="hidden" name="userId" value="101">
```

e) Evaluation

Technique	Advantages	Limitations
HttpSession	Data is stored on the server and is secure. Can store any Java object. Easy to use	Uses server memory. Lost when the session times out or the server restarts. Needs a session ID in cookie or URL
Cookies	Simple. Can stay for days (persistent) even after the browser is closed. No load on the server	Only small text data (about 4 KB). User can see, edit or delete it. Does not work if cookies are disabled
URL rewriting	Works even when cookies are disabled. No extra software needed	Long and ugly URLs. The ID is visible and can leak through sharing or logs. Every link must be encoded
Hidden fields	Simple and works in all browsers	Works only with forms. Data visible in the page source. Not persistent

f) Persistence, security and use cases

Considering persistence, only persistent cookies can remember a user for a long time. A session lasts only for the visit, usually until logout or a time out of 20 to 30 minutes. For security, HttpSession is the best because the data stays on the server, whereas cookies, URL rewriting and hidden fields keep the data on the client side. Session IDs can be stolen by session hijacking, so HTTPS, the HttpOnly flag and a new session ID after login should be used. About use cases: HttpSession is suitable for the login status and the shopping cart, cookies for the remember me option and user preferences, URL rewriting as a backup when cookies are blocked, and hidden fields for multi step forms.

5. Cookies vs Sessions - Critical Analysis

Question: Analyze and compare Cookies and Sessions in web applications. Present real-time use cases.

Focus Areas: Storage location, Security risks, Performance considerations

Answer

a) Comparison

Point	Cookies	Sessions (HttpSession)
Storage location	Client side, in the browser	Server side, in the memory of the server (only the ID is on the client)
Data size and type	About 4 KB, text only	Any Java object, limited only by the server memory
Life time	Set by setMaxAge(), can be days or months	Until logout, time out or the browser is closed
Security	Less secure, can be read, changed or stolen	More secure, the data is not seen by the user
Performance	No load on the server, but sent with every request	Uses server memory and grows with the number of users
Scalability	Scales easily	Needs session replication or sticky sessions in a cluster
Dependency	Fails if the user disables cookies	Normally uses a cookie for the session ID

b) Storage location

A cookie is stored in the browser of the user, so it remains available even after the server restarts. A session is stored on the server and the browser holds only the JSESSIONID. This is the basic difference from which the other differences come.

c) Security risks

Cookies can be viewed and modified by the user, and can be stolen by cross site scripting (XSS) or by sniffing on an unsecured network, so passwords or personal details should never be kept in them. The Secure and HttpOnly flags reduce these risks. Sessions are safer, but they have the risk of session hijacking and session fixation if the session ID is stolen or reused. To avoid this, the session should be invalidated on logout, a new ID should be created after login, and HTTPS should be used.

d) Performance considerations

Cookies do not use any memory on the server, but all of them are sent in the header of every request, so many large cookies increase the network traffic. Sessions keep the data in the server memory, so a site with a large number of users needs more memory, and the session time out should be kept reasonable. In a cluster of servers, the sessions must be shared or the user must always be sent to the same server.

e) Real-time use cases

Application	Technique used	Reason
Online shopping cart (Amazon, Flipkart)	Session	Cart items and login status are kept safely on the server
Net banking after login	Session	Sensitive data, and the session ends on logout or time out
Remember me on a login page	Persistent cookie	Keeps the user name for many days
Language or dark mode setting of a website	Cookie	Small, non-sensitive preference
Visited products or recently viewed items	Cookie	Remembers the user even after the browser is closed
Online exam portal	Session	Time limit and the answers are controlled by the server

In real projects both are used together. The session keeps the important data on the server, and a cookie carries the session ID and the small preferences of the user.

Code of Conduct

I K Ganesh / 192411170 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Ganesh

Rubrics for Calculation

Technical Presentation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness